



수학 시뮬레이터 제작 보고서

수행평가

《시뮬레이션 파일 저장하는 방법》

- ① 최종 시뮬레이터의 html 코드를 전체 복사한다.
- ② 윈도우 메모장을 켜 후 위의 코드를 붙여넣기 한다.
- ③ 메모장에서 파일 - 다른 이름으로 저장 - 각각 본인이름1.html, 본인이름2.html으로 저장
(예 : 이현진1.html, 이현진2.html으로 꼭 확장자 붙여서 저장)

[첫 번째 시뮬레이터 제작 보고서]

1. 내가 선택한 첫 번째 문제 (캡처해서 붙여넣기 해도 됨)

문제 1 2024학년도 연세대 기술 재확인 CHECK ☐ ☐ ☐

좌표평면 위의 점 P 와 평면벡터 $\vec{v}$ 에 대하여, $\overrightarrow{OQ} = \overrightarrow{OP} + t\vec{v} (t \geq 0)$ 을 만족하는 점 Q 가 그리는 반직선을 생각하자. 네 점 $P_1(6, 4)$, $P_2(1, 5)$, $P_3(0, 3)$, $P_4(0, -1)$ 과 네 벡터 $\vec{v}_1 = (-1, 0)$, $\vec{v}_2 = (0, -1)$, $\vec{v}_3 = (1, -1)$, $\vec{v}_4 = (1, 1)$ 에 대하여 $\overrightarrow{OQ_i} = \overrightarrow{OP_i} + t\vec{v_i} (i = 1, 2, 3, 4)$ 를 만족시키는 Q_i 가 그리는 반직선 l_i 가 주어져 있다. 다음 재시문을 참고하여 물음에 답하시오.

두 개의 반직선이 만났을 때, 양의 실수 a 에 대하여 다음 규칙을 따라 새로운 반직선들이 생성된다고 하자.
(가) 서로 다른 두 반직선 $l_i, l_j (1 \leq i, j \leq 4)$ 가 시작점 P_i 또는 P_j 가 아닌 점 P 에서 만날 때,
 $\overrightarrow{OR} = \overrightarrow{OP} + t(a\vec{v_i} + \vec{v_j}) (t \geq 0)$ 을 만족시키는 점 R 이 그리는 반직선과 $\overrightarrow{OS} = \overrightarrow{OP} + t(\vec{v_i} + a\vec{v_j}) (t \geq 0)$ 을 만족시키는 점 S 가 그리는 반직선이 각각 생성된다.
(나) 새로 생성된 반직선들을 포함하여 임의의 서로 다른 두 반직선이 시작점이 아닌 점에서 만날 때에도 (가)의 규칙이 계속 적용된다.

(1) 두 반직선 l_1, l_2 에 대하여 (가)에 따라 새로운 반직선들이 생성될 때, 점 $A(2, 2)$ 를 지나는 반직선이 생성될 수 있는 양의 실수 a 가 존재하면 그 값을 찾으시오. 만약 존재하지 않으면 그 이유를 설명하시오.

(2) 네 반직선 l_1, l_2, l_3, l_4 에 대하여 [재시문]의 (가), (나)에 따라 새로운 반직선들이 생성될 때, 점 $A(2, 2)$ 를 지나는 반직선이 생성될 수 있는 양의 실수 a 가 존재하면 그 값을 찾으시오. 만약 존재하지 않으면 그 이유를 설명하시오.

2. 이 문제의 풀이 (반드시 손글씨로 작성하여 캡처하거나 패드에 손글씨로 작성한 것을 붙여넣으세요.)

40

①) 평행의 2 직선과 2 직선, 접선의 개수는 $P(x) = 3x^2 + 2ax + b$,

$$3x^2 + 2ax + b = m \rightarrow 3x^2 + 2ax + (b-m) = 0,$$

$$\therefore D = a^2 - 3(b-m) \rightarrow \begin{cases} D > 0, \text{ 즉 } m > \frac{3b-a^2}{3} \text{ 일때 } 2 \text{ 개} \\ D = 0, \text{ 즉 } m = \frac{3b-a^2}{3} \text{ 일때 } 1 \text{ 개} \\ D < 0, \text{ 즉 } m < \frac{3b-a^2}{3} \text{ 일때 } 0 \text{ 개} \end{cases}$$

②) $3x^2 + 2ax + b = 0$ 이 두 실근 t_1, t_2 를 가진다고 하자. ($t_1 < t_2$)

$$\text{근과 계수와의 관계에 의해 } t_1 + t_2 = -\frac{2}{3}a$$

접선선은 $P_1 = (t_1, f(t_1))$ 에 접한다. $\therefore L_1$ 의 방정식을 구해보면

$$f(x) - g_1(x) = (x - t_1)^2 \cdot (x - g_1) \quad (g_1 \text{ 은 } Q_1 \text{ 의 x좌표})$$

$$\therefore a = -2t_1 - g_1, \quad \therefore g_1 = -a - 2t_1, \quad \text{같은 방법으로 } g_2 = -a - 2t_2$$

L_1 의 기울기가 m , P_1, Q_1 모두 L_1 위에 있다.

$$\begin{aligned} \therefore \overline{P_1 Q_1} &= |g_1 - t_1| \cdot \sqrt{1+m^2} = |-a - 3t_1| \cdot \sqrt{1+m^2} \\ &= \left| \frac{3t_2 - t_1}{2} \right| \sqrt{1+m^2} \end{aligned}$$

$$\begin{aligned} \overline{P_2 Q_2} &= |g_2 - t_2| \sqrt{1+m^2} = |-a - 3t_2| \cdot \sqrt{1+m^2} \\ &= \left| \frac{3t_1 - 3t_2}{2} \right| \sqrt{1+m^2} \end{aligned}$$

$$\left| \frac{3t_2 - 3t_1}{2} \right| = \left| \frac{3t_1 - 3t_2}{2} \right| = \frac{3(t_2 - t_1)}{2}$$

$$\therefore \overline{P_1 Q_1} = \overline{P_2 Q_2}$$

3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유를 서술하세요.

이 문제는 반직선이 한 번만 생성되는 것이 아니라, 새로 생긴 반직선들이 다시 만나면서 계속 새로운 반직선을 만드는 구조이다. 따라서 종이에 몇 개의 그림만 그려서는 전체 과정을 직관적으로 파악하기 어렵다.

또한 a 값에 따라 새 반직선의 방향이 달라지므로, a 를 바꾸어 가며 점 $A(2,2)$ 를 지나는 반직선이 생기는지 확인할 필요가 있었다. 특히 풀이에서 중요한 $\vec{w} \cdot \vec{PA} > 0$

이라는 성질이 실제로 계속 유지되는지 시각적으로 확인하기 위해 시뮬레이터가 필요하다고 느꼈다.

4. 사용한 AI 플랫폼을 모두 적으세요.

ChatGPT

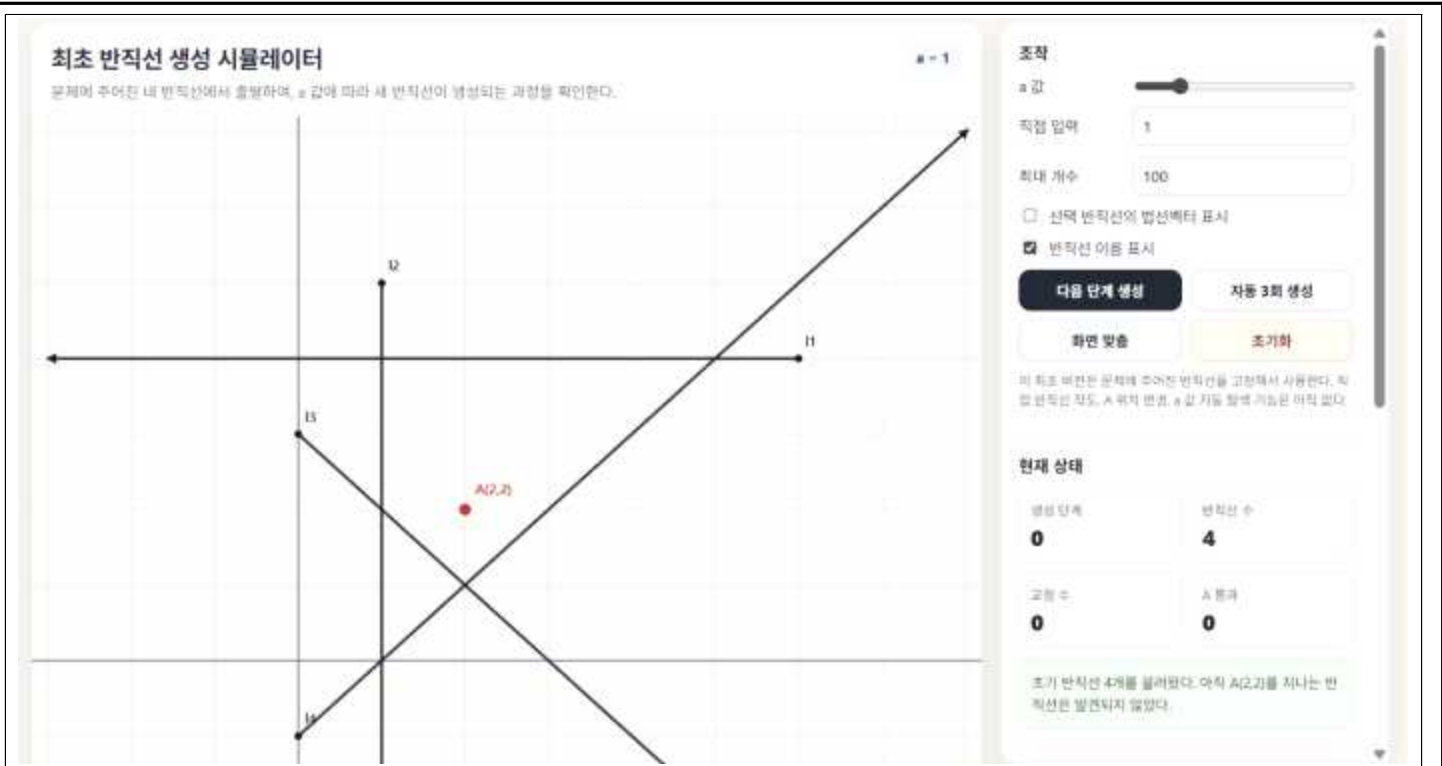
5. 시뮬레이터를 만들기 위한 최초의 프롬프트를 작성하세요.

위 문제의 반직선 생성 규칙을 바탕으로 HTML 시뮬레이터를 만들어줘.

처음 주어진 네 반직선 l_1, l_2, l_3, l_4 와 점 $A(2,2)$ 를 좌표평면에 표시하고, a 값을 조절하면 반직선들이 만나는 점에서 새 반직선이 생성되도록 해줘.

또한 각 반직선이 점 A 를 지나는지 판정하고, 선택한 반직선의 방향벡터, 법선벡터, $w \cdot PA$ 값을 표시해줘.

6. 최초 시뮬레이터의 모습을 캡처해서 붙여넣으세요.



7. 최초 시뮬레이터에서 수정해야 할 사항을 서술하세요.

최초 시뮬레이터는 문제에 주어진 네 반직선을 표시하고, a 값에 따라 새 반직선을 생성하는 기능은 잘 구현되었다. 하지만 다음과 같은 보완이 필요했다.

첫째, 사용자가 직접 반직선을 좌표평면에 찍어 추가하는 기능이 없었다.

둘째, 목표점 A 도 직접 설정할 수 없었다.

셋째, 가능한 a 값을 자동으로 찾는 기능이 필요했다.

넷째, 반직선 수가 많아질 때를 대비해 중복 제거와 최대 개수 제한 기능이 필요했다.

다섯째, `view.minX` 오류가 발생하여 좌표평면 범위가 설정되기 전에도 안전하게 실행되도록 수정이 필요했다.

8. 수정하기 위한 프롬프트를 (2번째 ~ n번째) 모두 작성하세요.

2번째 프롬프트

원하는 반직선들과 점을 찍으면 이를 만족하는 a 값을 찾는 시뮬레이터도 만들 수 있어? 가능하면 어떻게 구현할지 구조와 계획을 작성해줘.

3번째 프롬프트

반직선은 점이나 벡터 입력보다는 직접 좌표평면에 찍어서 그릴 수 있게 만들고 싶어. 가능할까?

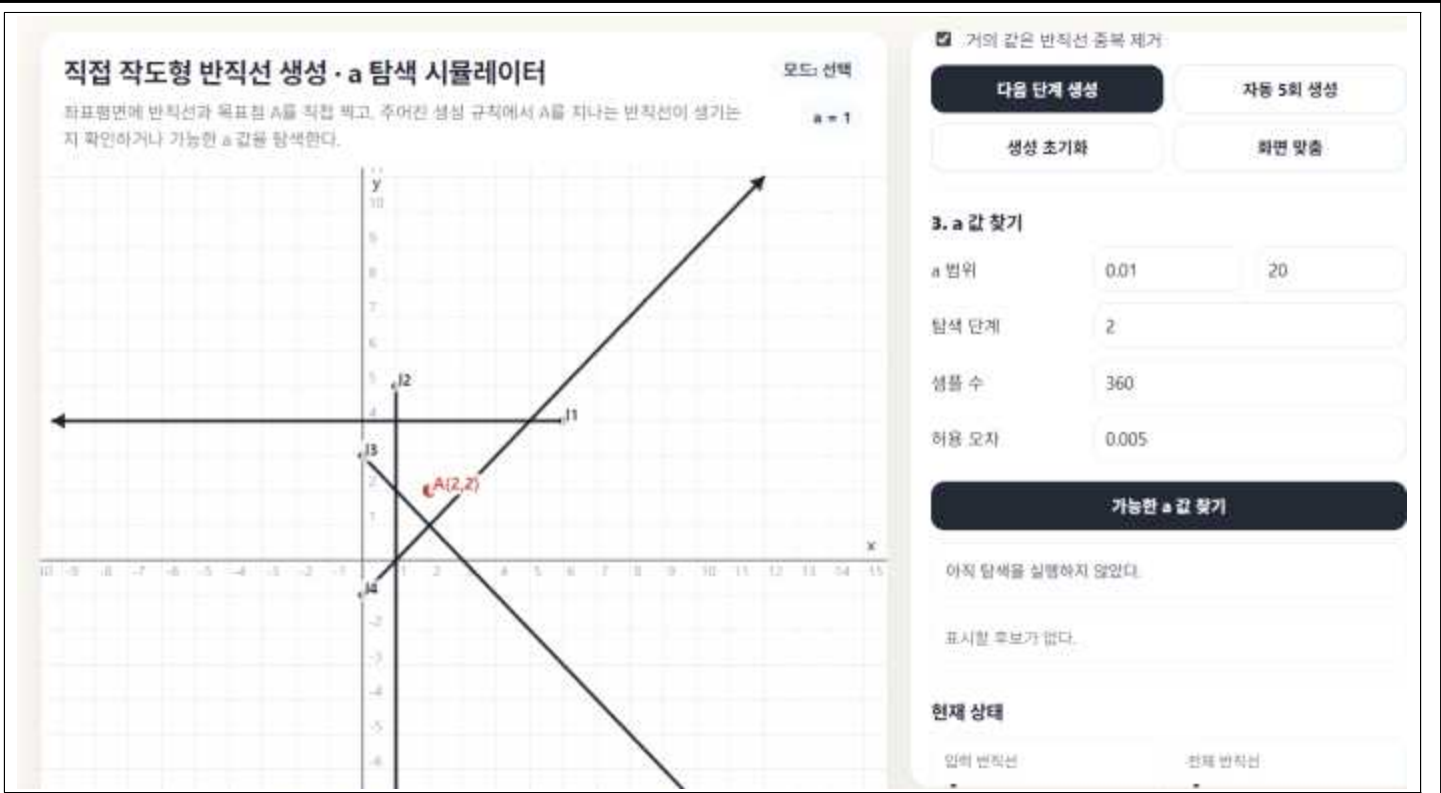
4번째 프롬프트

위 형태로 코드를 작성해줘. 좌표평면에서 직접 반직선을 그리고, 목표점 A 도 직접 찍을 수 있으며, 가능한 a 값을 찾는 기능까지 포함해줘.

5번째 프롬프트

`TypeError: Cannot read properties of undefined (reading 'minX')` 에러가 생겼어. 에러가 생긴 이유를 찾고 다른 오류들은 없는지도 체크해서 코드를 수정해줘.

9. 최종 시뮬레이터의 모습을 캡처해서 붙여넣으세요.



10. 자신이 만든 시뮬레이터를 처음 본 사람이 사용할 수 있도록 사용 방법을 서술하세요.

먼저 문제 예시 불러오기 버튼을 누르면 문제의 네 반직선과 점 $A(2,2)$ 가 자동으로 표시된다. 이후 a 값을 슬라이더나 직접 입력으로 조절하고, 다음 단계 생성 버튼을 누르면 반직선들이 만나 새 반직선이 생성된다. 자동 5회 생성 버튼을 누르면 여러 단계를 한 번에 확인할 수 있다. 반직선을 클릭하면 오른쪽 패널에서 시작점, 방향벡터, 법선벡터, $\vec{w} \cdot \vec{PA}$ 값을 확인할 수 있다. 직접 반직선을 만들고 싶으면 반직선 추가 모드를 선택한다. 첫 번째 클릭은 시작점 P , 두 번째 클릭은 방향점 Q 가 되며, 방향벡터는 자동으로 $Q-P$ 로 계산된다. 목표점 A 를 바꾸고 싶으면 A 찍기 모드를 선택한 뒤 원하는 위치를 클릭하면 된다. 가능한 a 값을 찾고 싶으면 a 값 찾기 영역에서 탐색 범위와 단계를 설정한 뒤 가능한 a 값 찾기 버튼을 누른다. 후보 a 값을 클릭하면 그 값으로 시뮬레이션이 재생된다.

11. 시뮬레이터가 자신이 의도한대로 문제를 파악하거나 풀이과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

시뮬레이터는 문제의 반복 생성 구조를 이해하는 데 도움이 되었다. 반직선들이 만날 때마다 새 반직선이 생기는 과정을 직접 볼 수 있었기 때문에, 단순한 그림보다 문제 상황을 훨씬 쉽게 파악할 수 있었다. 또한 풀이에서 사용한 $\vec{w} \cdot \vec{PA} > 0$ 이라는 성질을 확인하는 데 도움이 되었다. 반직선을 클릭하면 법선벡터와 $\vec{w} \cdot \vec{PA}$ 값이 표시되므로, 점 A 가 반직선 위에 놓이지 않는 이유를 시각적으로 검토할 수 있었다. 하지만 시뮬레이터는 수학적 증명을 대신해주지는 못한다. 특히 모든 양의 실수 a 와 무한한 반복 과정을 완전히 확인하는 것은 불가능하다. 따라서 시뮬레이터는 풀이 방향을 잡고 계산을 검토하는 보조 도구로 사용하고, 최종 결론은 법선벡터와 내적 부호 유지 성질을 이용해 직접 증명해야 한다.

[두 번째 시뮬레이터 제작 보고서]

1. 내가 선택한 두 번째 문제 (캡처해서 붙여넣기 해도 됨)

xy 평면에 3차 곡선 $C: y = x^3 - x$ 와 점 P 가 있다.

- (1) P 를 지나는 C 의 접선이 한 개가 되는 P 의 영역을 도식하라.
- (2) P 를 지나는 C 의 접선이 세 개가 되는 P 의 영역을 D 라 한다. P 가 D 의 x 축 위를 움직일 때, P 가 지나는 C 의 세 개의 접선의 접점을 꼭짓점으로 하는 삼각형의 무게중심의 자취의 방정식을 구하라.

2. 이 문제의 풀이 (반드시 손글씨로 작성하여 캡처하거나 패드에 손글씨로 작성한 것을 붙여넣으세요.)

(1) 두 반직선 l_1, l_2 는 $P(1,4)$ 에서 만난다. 이때, $A(2,2)$.

$\vec{PA} = \langle 1, -2 \rangle$. 조건에 따라 $\frac{\alpha \vec{v}_1 + \vec{v}_2}{|\vec{v}_1 + \alpha \vec{v}_2|}$ 의 반직선이 생김.

$\therefore \vec{v}_1 = \langle -\alpha, -1 \rangle \rightarrow$ 방향 벡터: $\vec{w}_1 = \langle 1, -\alpha \rangle$.

$\therefore \vec{w}_1 \cdot \vec{PA} = 4 - 2\alpha > 0 \therefore$ 반직선은 A 위로 지나지 않는다.

$\vec{v}_2 = \langle -1, -\alpha \rangle$ 역시 $\vec{w}_2 = \langle \alpha, -1 \rangle$

$\vec{w}_2 \cdot \vec{PA} = \alpha - 2 > 0 \therefore$ 역시 반직선은 A 위로 지나지 않으므로 존재하지 않는다.

(2) (1)처럼 l_1, l_2, l_3, l_4 에 대해 $\vec{w}_i, \vec{PA}$ 를 계산하자.

$P_1 = \langle 6, 4 \rangle, \vec{v}_1 = \langle -1, 0 \rangle \therefore \vec{w}_1 = \langle 0, -1 \rangle$.

$\therefore \vec{w}_1 \cdot \vec{PA} = \langle -4, -2 \rangle \therefore \vec{w}_1 \cdot \vec{PA} = 2 > 0$.

$P_2 = \langle 1, 5 \rangle, \vec{v}_2 = \langle 0, -1 \rangle \therefore \vec{w}_2 = \langle 1, 0 \rangle$.

$\vec{w}_2 \cdot \vec{PA} = 1 > 0$.

$P_3 = \langle 0, 3 \rangle, \vec{v}_3 = \langle 1, -1 \rangle \therefore \vec{w}_3 = \langle 1, 1 \rangle \therefore \vec{w}_3 \cdot \vec{PA} = 1 > 0$.

$P_4 = \langle 0, -1 \rangle, \vec{v}_4 = \langle 1, 1 \rangle, \vec{w}_4 = \langle 1, 1 \rangle, \vec{w}_4 \cdot \vec{PA} = 3 > 0 \therefore \vec{w}_4 \cdot \vec{PA} = 1 > 0$.

(2) $\therefore$ 처음 네 반직선 모두 $\vec{w}_i \cdot \vec{PA} > 0$ 은 만족한다.

여론 세로 생기는 반직선들에서도 선점들은 보아.

A 로 가는 두 반직선 m, m' 라 하면, 이들의 방향 벡터는 $\vec{v}_1, \vec{v}_2$ 라 하자, 대응적인 반직선 벡터는 $\vec{w}_1, \vec{w}_2$ 라 하면, 두 반직선의 교점은 P 라 한다.

$\vec{w}_1 \cdot \vec{PA} > 0, \vec{w}_2 \cdot \vec{PA} > 0$ 이 성립.

이때 세로 생성되는 방향 벡터는 $\vec{v} = \alpha \vec{v}_1 + \vec{v}_2$, $\therefore \vec{w} = \alpha \vec{w}_1 + \vec{w}_2$,
 $\frac{\alpha \vec{v}_1 + \vec{v}_2}{|\alpha \vec{v}_1 + \vec{v}_2|}$

$\therefore \vec{w} \cdot \vec{PA} = (\alpha \vec{w}_1 + \vec{w}_2) \cdot \vec{PA}$

$= \alpha (\vec{w}_1 \cdot \vec{PA}) + \vec{w}_2 \cdot \vec{PA} > 0$.

$\vec{v} = \vec{v}_1 + \alpha \vec{v}_2$ 에서도 역시 만족한다.

$\therefore$ 세로 생성되는 모든 반직선에서도 $\vec{w}_i \cdot \vec{PA} > 0$,

$\therefore A$ 는 세로 생성되는 반직선위에 존재할 수 없으므로 A 는 존재하지 않는다.

3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유를 서술하세요.

이 문제는 한 점 P 의 위치에 따라 곡선 $C: y = x^3 - x$ 에 그을 수 있는 접선의 개수가 달라지는 상황을 다룬다. 단순히 식으로만 보면 접선이 한 개가 되는 영역과 세 개가 되는 영역이 어떻게 나뉘는지 직관적으로 파악하기 어렵다.

첫째, 접선 개수의 변화를 시각적으로 확인할 필요가 있었다. 점 P 의 위치가 조금만 바뀌어도 접선이 한 개에서 세 개로 변하거나, 세 개에서 한 개로 변할 수 있다. 이를 종이 위의 정적인 그래프만으로 이해하기에는 한계가 있으므로, 점 P 를 직접 움직이며 접선 개수가 실시간으로 변하는 모습을 확인하는 시뮬레이터가 필요하다고 생각했다.

둘째, 영역 D 의 의미를 직관적으로 파악하기 위해 시뮬레이션이 필요했다. 풀이에서는 $(x + y)(x + y - x^3) < 0$ 이라는 부등식으로 접선이 세 개가 되는 영역을 구하지만, 이 부등식이 실제 좌표평면에서 어떤 부분을 의미하는지 바로 떠올리기 어렵다. 시뮬레이터에서 $y = -x$ 와 $y = x^3 - x$ 를 함께 표시하고, 두 곡선 사이의 영역을 색으로 나타내면 영역 D 를 훨씬 쉽게 이해할 수 있다.

셋째, 무게중심의 자취를 확인하는 데 도움이 된다. P 가 x 축 위를 움직일 때 세 접점이 계속 변하고, 이 세 접점이 만드는 삼각형의 무게중심도 함께 움직인다. 수식으로는 $y = 9x^3 - 2x$ 라는 자취가 나오지만, 시뮬레이터를 통해 실제로 무게중심이 이 곡선을 따라 움직이는지 확인하면 풀이 결과를 검토할 수 있다.

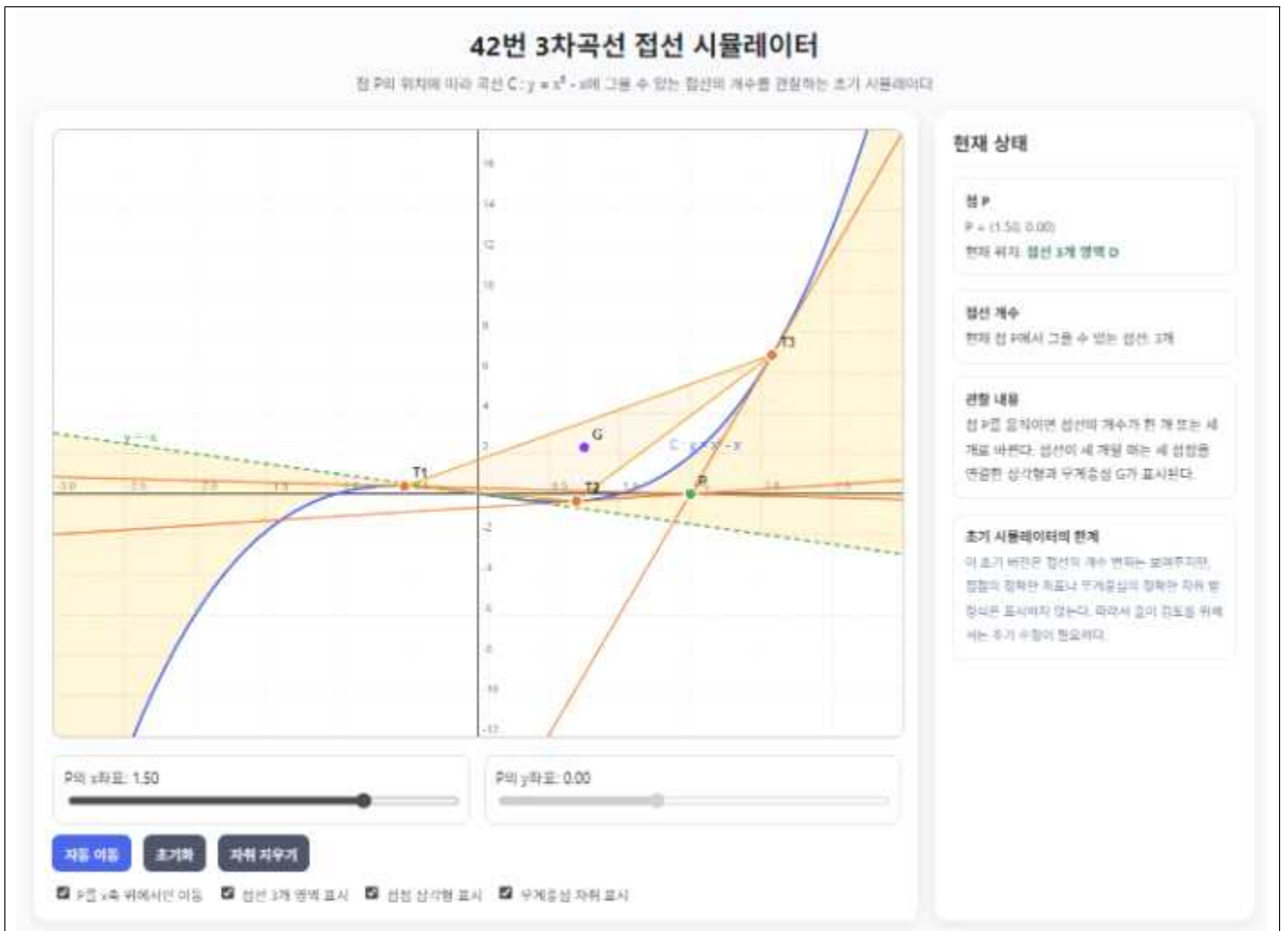
4. 사용한 AI 플랫폼을 모두 적으세요.

ChatGPT

5. 시뮬레이터를 만들기 위한 최초의 프롬프트를 작성하세요.

이 문제를 푸는 데 도움을 줄 수 있는 HTML 시뮬레이터를 만들어줘. 곡선 $C : y = x^3 - x$ 와 점 P 를 좌표평면 위에 표시하고, 점 P 를 마우스로 드래그하거나 슬라이더로 움직일 수 있게 해줘. 점 P 에서 곡선 C 에 그을 수 있는 접선을 실시간으로 표시하고, 접선의 개수가 한 개인지 세 개인지 오른쪽 패널에 나타나게 해줘. 접선이 세 개가 되는 영역 D 는 색으로 표시하고, 경계선인 $y = -x$ 와 $y = x^3 - x$ 도 함께 나타내줘. 또한 P 가 x 축 위를 움직이는 모드를 만들어서, 접선이 세 개일 때 세 접점을 연결한 삼각형과 그 무게중심 G 의 자취를 보여줘.

6. 최초 시뮬레이터의 모습을 캡처해서 붙여넣으세요.



7. 최초 시뮬레이터에서 수정해야 할 사항을 서술하세요.

최초 시뮬레이터는 점 P 를 움직일 때 접선의 개수가 변하는 모습과 접선이 그려지는 과정을 보여주었지만, 실제 문제 풀이에 직접적으로 활용하기에는 몇 가지 보완할 점이 있었다.

첫째, 접선이 세 개가 되는 영역 D 가 명확하게 표시되지 않았다. 문제의 (1)번은 접선이 한 개가 되는 P 의 영역을 도식하는 것이므로, 접선이 한 개가 되는 영역과 세 개가 되는 영역이 시각적으로 구분되어야 한다. 이를 위해 $y = -x$ 와 $y = x^3 - x$ 를 경계선으로 표시하고, 두 곡선 사이의 영역 D 를 색으로 칠하는 기능이 필요했다.

둘째, 접점의 좌표와 접선 개수를 숫자로 확인하는 기능이 부족했다. 시각적으로 접선이 여러 개 보이더라도, 정확

히 접선이 한 개인지 세 개인지, 각 접점의 좌표가 무엇인지 확인하기 어려웠다. 따라서 오른쪽 패널에 접선 개수와 접점 좌표를 표시하는 기능을 추가할 필요가 있었다.

셋째, (2)번에서 필요한 무게중심의 자취가 충분히 강조되지 않았다. 문제의 핵심은 P가 x축 위를 움직일 때 세 접점 삼각형의 무게중심이 어떤 방정식을 만족하는지 구하는 것이다. 따라서 P를 x축 위에서만 움직이는 모드, 접점 삼각형 표시, 무게중심 G 표시, 무게중심의 자취 표시 기능이 추가로 필요했다.

넷째, 최종적으로 구한 자취 $y = 9x^3 - 2x$ 와 실제 무게중심의 움직임을 비교할 수 있는 기능이 필요했다. 이를 통해 내가 구한 자취 방정식이 시뮬레이터의 결과와 일치하는지 확인할 수 있다.

8. 수정하기 위한 프롬프트를 (2번째 ~ n번째) 모두 작성하세요.

수정 프롬프트 1

접선이 세 개가 되는 영역 D를 더 명확하게 표시해줘. 경계선인 $y = -x$ 와 $y = x^3 - x$ 를 서로 다른 색으로 표시하고, 두 곡선 사이의 영역을 연한 색으로 칠해줘. 점 P가 현재 접선 한 개 영역에 있는지, 접선 세 개 영역 D에 있는지 오른쪽 패널에 표시해줘.

수정 프롬프트 2

점 P에서 곡선 C에 그을 수 있는 접선의 개수와 접점 좌표를 오른쪽 정보 패널에 표시해줘. 접선이 한 개일 때는 접점 하나만, 접선이 세 개일 때는 세 접점의 좌표를 모두 표시해줘. 접선 개수가 바뀌는 경계 위에 있을 때도 따로 표시해줘.

수정 프롬프트 3

2번 문항을 분석할 수 있도록 P를 x축 위에서만 이동하는 모드를 추가해줘. 이 모드를 켜면 P의 y좌표는 항상 0이 되게 하고, P가 x축 위에서 움직일 때 접선이 세 개가 되는 구간인 $|a| > 1$ 을 확인할 수 있게 해줘.

수정 프롬프트 4

접선이 세 개일 때 세 접점을 연결해서 삼각형을 만들고, 그 삼각형의 무게중심 G를 표시해줘. 또한 P가 x축 위를 움직일 때 무게중심 G의 자취가 남도록 만들어줘.

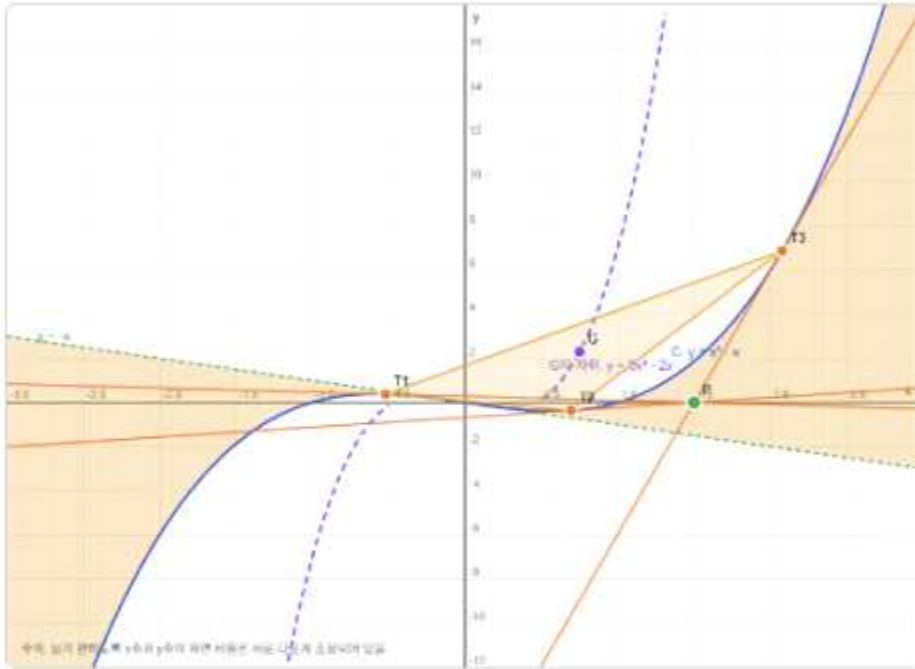
수정 프롬프트 5

내가 풀이에서 구한 무게중심의 자취 방정식 $y = 9x^3 - 2x$ 를 그래프 위에 함께 표시해줘. 실제 시뮬레이터에서 찍히는 무게중심 G의 자취와 이론적으로 구한 자취가 일치하는지 비교할 수 있게 해줘.

9. 최종 시뮬레이터의 모습을 캡처해서 붙여넣으세요.

42번 3차곡선 접선 시뮬레이터

곡선 $C: y = x^3 - x$ 와 점 P 에 대하여, P 에서 C 에 그릴 수 있는 접선의 개수와 접점 삼각형의 무게중심 위치를 시각화하는 시뮬레이터



두 점의 위치 관계에 따라 x축과 y축의 좌변/우변에 서로 다른 개수의 접선이 있을 수 있다.

P의 x좌표 a: 1.500



P의 y좌표 b: 0.000



자동 해동 시작

무계중심 자취 스캔

P 조거

자취 지우기

☒ P를 x축 위에세면 이동

☒ 경계선 표시

☒ 접점 삼각형 표시

☒ 이론 자취 $y = 9x^2 - 2x$ 표시

☒ 접선 3개 영역 D 표시

☒ 접선 표시

☒ 무게중심 자취 표시

☒ 점 이동 표시

현재 상태

점 P

$P = (1.500, 0.000)$

현재 영역: 접선 3개 영역 D 내부

접선 3개 영역 D는 $y = -x$ 와 $y = x^2 - x$ 사이의 영역으로 볼 수 있다.

접선 개수

판별식 $\Delta = -0.125281$

서로 다른 접선 개수 = 3개

접점의 좌표와 t는

$$2t^3 - 3at^2 + a + b = 0$$

의 실근이다.

접점

$T1 = (-0.5203, 0.8795)$

$T2 = (0.0943, -0.3580)$

$T3 = (2.0760, 6.8708)$

각 접점과 $T = (0, t^3 - t)$ 로 표시한다.

접점 삼각형과 무게중심

$G = (0.7500, 2.2969)$

x축 오드 조건: $P = (a, 0)$, $|a| > 1$ 조건 만족

$P = (a, 0)$, $|a| > 1$ 일 때

$$9a = a/2$$

$$9y = 9a^2/8 - a$$

$$\text{파라볼라 } y = 9x^2 - 2x$$

사용 방법

점 P를 드래그하거나 슬라이더를 움직이면 P에서 곡선 C에 그릴 수 있는 접선들이 실시간으로 표시된다. x축 미동 오드 일 때면 2번 판정점 P가 x축 위에세면 움직이며, 접선이 3개일 때 세 접점을 연결한 삼각형과 무게중심 G가 표시된다.

10. 자신이 만든 시뮬레이터를 처음 본 사람이 사용할 수 있도록 사용 방법을 서술하세요.

기본 그래프 확인

화면 왼쪽에는 좌표평면과 곡선 $C : y = x^3 - x$ 가 표시되어 있다. 또한 접선 개수가 변하는 경계선인 $y = -x$ 와 $y = x^3 - x$ 가 함께 표시되어 있다.

점 P 움직이기

그래프 위의 점 P를 마우스로 드래그하거나, 아래쪽 슬라이더를 움직이면 P의 위치를 바꿀 수 있다. P의 위치가 바뀌면 P에서 곡선 C에 그을 수 있는 접선들이 자동으로 다시 계산되어 화면에 표시된다.

접선 개수 확인

오른쪽 정보 패널에서 현재 점 P에서 그을 수 있는 접선의 개수를 확인할 수 있다. P가 영역 D 안에 있으면 접선이 세 개 표시되고, 영역 D 밖에 있으면 접선이 한 개 표시된다.

영역 D 확인

접선 3개 영역 D 표시 기능을 켜면 접선이 세 개가 되는 영역이 색으로 나타난다. 이 영역은 두 경계선 $y = -x$ 와 $y = x^3 - x$ 사이의 부분이다.

x축 이동 모드 사용

P를 x축 위에서만 이동 기능을 켜면 점 P가 x축 위에서만 움직인다. 이 기능은 문제의 (2)번 조건을 확인하기 위한 것이다. 이 상태에서 P를 $x > 1$ 또는 $x < -1$ 인 위치로 움직이면 접선이 세 개가 된다.

접점 삼각형과 무게중심 확인

접선이 세 개일 때 세 접점이 표시되고, 이 접점들을 연결한 삼각형이 나타난다. 동시에 삼각형의 무게중심 G가 표시된다.

무게중심 자취 확인

P를 x축 위에서 움직이면 무게중심 G의 자취가 화면에 남는다. 이론 자취 $y = 9x^3 - 2x$ 표시 기능을 켜면 풀이에서 구한 자취와 실제 시뮬레이션 결과를 비교할 수 있다.

11. 시뮬레이터가 자신이 의도한대로 문제를 파악하거나 풀이과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

이 시뮬레이터는 점 P의 위치에 따라 접선의 개수가 달라지는 상황을 시각적으로 이해하고, 무게중심의 자취를 확인하는 데 도움이 되었다.

[도움이 된 부분]

첫째, 접선 개수의 변화를 직관적으로 파악할 수 있었다. 문제를 식으로만 보면 왜 어떤 영역에서는 접선이 한 개이고, 어떤 영역에서는 접선이 세 개인지 이해하기 어렵다. 하지만 시뮬레이터에서 점 P를 움직여 보니, P가 두 경계선 $y = -x$ 와 $y = x^3 - x$ 사이에 있을 때 접선이 세 개가 되고, 그 바깥쪽에서는 접선이 한 개가 되는 것을 직접 확인할 수 있었다.

둘째, 영역 D의 형태를 이해하는 데 도움이 되었다. 풀이에서는 접선이 세 개가 되는 조건이 $(x + y)(x + y - x^3) < 0$ 으로 나오지만, 이 부등식만 보고 영역을 정확히 상상하기는 쉽지 않았다. 시뮬레이터에서 영역 D가 색으로 표시되면서, D가 두 곡선 사이에 있는 영역이라는 것을 명확하게 파악할 수 있었다.

셋째, (2)번 풀이의 핵심인 무게중심의 자취를 검토할 수 있었다. P를 x축 위에서 움직이면 세 접점으로 이루어진 삼각형이 계속 변하고, 그 무게중심 G도 함께 움직인다. 시뮬레이터에서 G의 자취가 $y = 9x^3 - 2x$ 위에 나타나는 것을 확인하면서, 풀이에서 구한 자취 방정식이 타당하다는 것을 검토할 수 있었다.

넷째, 접점의 변화 과정을 이해하는 데 도움이 되었다. P가 움직이면 접선의 접점들도 곡선 위에서 함께 이동한다. 이를 통해 접점의 x좌표를 t로 두고 접선 조건식을 세우는 풀이 과정이 단순한 계산이 아니라 실제 기하적 상황을 반영한다는 것을 이해할 수 있었다.

[한계점]

그러나 시뮬레이터가 수학적 증명을 완전히 대신해주지는 못했다. 예를 들어 시뮬레이터는 접선이 세 개가 되는 영역을 색으로 보여주지만, 왜 그 영역이 $(x + y)(x + y - x^3) < 0$ 으로 표현되는지는 직접 접선 방정식을 세우고 삼차방정식의 실근 개수를 분석해야 알 수 있다.

또한 시뮬레이터는 컴퓨터 계산을 이용하기 때문에 접점 좌표와 무게중심 좌표가 대부분 소수 근삿값으로 표시된다. 따라서 정확한 자취 방정식 $y = 9x^3 - 2x$ 를 얻기 위해서는 근과 계수의 관계를 이용한 대수적 풀이가 반드시 필요하다.

[정리]

결론적으로 시뮬레이터는 문제 상황을 이해하고, 영역 D와 무게중심 자취를 시각적으로 확인하는 데 큰 도움이 되었다. 다만 최종 답을 엄밀하게 도출하기 위해서는 접선 방정식, 삼차방정식의 실근 조건, 근과 계수의 관계를 이용한 수학적 풀이가 필요했다.

[마무리]

1. 두 문제에 대한 시뮬레이터를 기획하고 활용하여 문제를 해결하는 전 과정에서, 생성형 AI가 제공할 수 있는 가치와 반드시 인간(학생 본인)이 주도해야 하는 수학적 사고의 영역을 구분하여 서술하세요.

두 개의 시뮬레이터를 제작하며 생성형 AI를 통해 문제 상황을 자세하고, 구체적으로 시각화 할 수 있고, 문제의 조건을 다양하게 변화시켜나가며 문제에서 요구하는 내용이 무엇인지 이해하는데 큰 도움을 주었으며, HTML 코드를 통해 이를 구현하는데 사용되었다. 그러나, AI가 시뮬레이터를 만들 수 있다해도, 수학적인 풀이를 통한 증명이나, 풀이가 설명되지 않는다는 문제의 핵심적인 요건을 해석하고, 이러한 성질들을 확인하고, 활용하면서 시뮬레이터를 통해 확인했던 현상들을 수학적으로 증명하는 과정은 직접 했어야 했다. 이러한 과정으로 AI는 시각화와 구현을 도울 수 있는 도구이나, 문제 자체를 이해하고, 풀이하면서 증명하는 과정과 물리를 이해하는 과정들은 사람이 책임져야 한다고 생각했다.

2. 복잡한 수학 문제를 AI와 협업하여 시뮬레이터로 구현해 본 경험이, 향후 자신이 희망하는 진로 분야에서 실무적인 문제를 해결하는 데 어떤 시사점을 주는지 서술하세요.

복잡한 수학 문제를 AI와 함께 시뮬레이터로 구현하면서, 실제 공학적 문제 해결에서도 계산과 시각화 도구를 적절히 활용하는 능력이 중요하다는 것을 느꼈다. 화학 공학과 같은 분야들에서도 실험결과 이론적인 모델일 단순히 식이나 정보로 이해하는 것이 아니라, 그래프나 시뮬레이션 프로그램들을 직접 AI를 활용하며 만들어보면서 변화 양상이나, 변수들에 의해 변화되는 현상을 쉽게 파악 할 수 있을 것이다.

또한 AI가 코드를 작성해 줄 수 있더라도, 복잡한 문제들의 경우 사용자가 문제의 조건과 목적과 구현해야하는 내용들에 대해 정확히 전달하지 못해, AI가 이해하지 못하면 잘못된 시뮬레이터가 만들어 질 수 있다는 사실도 경험하면서 복잡한 실무적인 문제들을 해결할 때에도 AI를 단순히 답을 내는 도구로 사용하는 것이 아니라, 직접 세운 가설과 계산 과정을 검토하고 시각화할 수 있는 도구로써 다양한 분석에 활용할 수 있음을 깨닫게 되었다.

3. 보고서를 작성하며 느낀점을 간단히 작성하세요.

이번 보고서를 작성하면서 시뮬레이터는 단순히 보기 좋은 프로그램은 아니라, 수학적 성질을 표현하고 시각적으로 쉽게 이해하기 위해 필요한 보조적으로 사용가능함을 깨닫게 되었고, 처음에는 문제를 코드로 구현하여 시뮬레이터를 만드는 것에만 집중하였지만, 보고서를 작성하면서 이 시뮬레이터들이 왜 필요한지 생각해보고, 어떠한 성질을 확인하기 위한 것인지 설명하는 과정역시 매우 중요함을 깨닫게 되었다. 또한 시뮬레이터로 관찰한 결과는 현상의 증명이 아니기에, 시각적인 확인이나 시뮬레이션과 수학적 증명과정은 별개의 과정임을 깨닫고, AI와 협업하여 코드를 수정하고 오류를 해결하는 과정을 통해 AI를 활용하여 복잡한 여러 문제들을 해결하고, 이를 위해 핵심 아이디어나, 정보를 논리적으로 정리하는 과정을 스스로 수행하면서 문제를 더 깊이 이해하는 과정을 경험할 수 있게 되었다.